UAV Propeller Performance — Source Code

All analysis, machine-learning and SQL source for the capstone. Files are listed in execution order.

Contents

1. config.py
2. 01_data_preparation.py
3. 02_solidity_analysis.py
4. 03_eda_visualizations.py
5. 04_ml_models.py
6. 05_run_sql.py
7. 06_tableau_dashboard.py
8. 07_build_submission.py
9. run_all.py
10. week1_queries.sql

config.py

```
 """Shared paths and helpers for the propeller-performance pipeline."""
import os
import re

HERE = os.path.dirname(os.path.abspath(__file__))
RAW = os.path.join(HERE, "..", "data", "raw")
PROCESSED = os.path.join(HERE, "..", "data", "processed")
FIGURES = os.path.join(HERE, "..", "outputs", "figures")
TABLES = os.path.join(HERE, "..", "outputs", "tables")

for _d in (PROCESSED, FIGURES, TABLES):
    os.makedirs(_d, exist_ok=True)


def to_identifier(name: str) -> str:
    """Convert a human column label into a valid, PEP 8-style Python identifier.

    'Propeller's Name'                 -> 'propeller_name'
    'Adimensional Chord - c/R'         -> 'adimensional_chord_c_r'
    'beta - Angle Relative to Rotation'-> 'beta_angle_relative_to_rotation'
    """
    s = name.strip().lower()
    s = s.replace("'s", "")            # drop possessive 's
    s = re.sub(r"[^0-9a-z]+", "_", s)  # non-alphanumerics -> underscore
    s = re.sub(r"_+", "_", s).strip("_")
    if s and s[0].isdigit():
        s = "_" + s
    return s
```

01_data_preparation.py

```
 """
01_data_preparation.py  -  Week 2 (Data Science), Part 1
=========================================================
Tasks addressed
---------------
1. Read every raw file and APPEND the versions, producing two tidy datasets:
   one experiment dataset and one blade-geometry dataset.
2. Rename all variables to follow the Python identifier naming convention.
3. Convert the a-dimensional radius (r/R) and a-dimensional chord (c/R) into
   physical RADIUS and CHORD distributions by multiplying with the blade
   radius R = Diameter / 2 (for every blade of every propeller).

Data note: the experiment data ships in three volumes (Experiment_vol1..3) and
the geometry data in two volumes (Geom_vol1..2). We read whatever volume files
are present for each prefix, so the pipeline is robust to the number of parts.

Outputs (data/processed/):
   experiment_all.csv     - appended + renamed experiment data
   geometry_all.csv       - appended + renamed geometry data + radius/chord dist
"""
import glob
import numpy as np
import pandas as pd

from config import RAW, PROCESSED, to_identifier
import os


def load_and_append(prefix):
    """Read every <prefix>_vol*.csv present in the raw folder and append them."""
    paths = sorted(glob.glob(os.path.join(RAW, f"{prefix}_vol*.csv")))
    if not paths:
        raise FileNotFoundError(f"No raw files matching {prefix}_vol*.csv in {RAW}")
    frames = []
    for path in paths:
        vol = int(os.path.basename(path).split("vol")[1].split(".")[0])
        df = pd.read_csv(path)
        df["source_volume"] = vol          # keep provenance of each row
        frames.append(df)
        print(f"  read {os.path.basename(path)}  ->  {df.shape[0]} rows, {df.shape[1]-1} cols")
    combined = pd.concat(frames, ignore_index=True)
    print(f"  appended -> {combined.shape[0]} rows")
    return combined


def rename_to_identifiers(df):
    """Rename every column to a valid Python identifier and report the map."""
    mapping = {c: to_identifier(c) for c in df.columns}
    print("  column rename map:")
    for k, v in mapping.items():
        print(f"    {k!r:45s} -> {v}")
    return df.rename(columns=mapping)
```

```
def add_physical_distributions(geom):
    """radius = (r/R) * R  and  chord = (c/R) * R, with R = diameter / 2."""
    R = geom["propeller_diameter"] / 2.0
    geom["blade_radius_R"] = R
    geom["radius_distribution"] = geom["adimensional_radius_r_r"] * R
    geom["chord_distribution"] = geom["adimensional_chord_c_r"] * R
    return geom


def main():
    print("STEP 1  Reading & appending EXPERIMENT workbooks")
    experiment = load_and_append("Experiment")
    print("\nSTEP 2  Renaming EXPERIMENT columns to Python identifiers")
    experiment = rename_to_identifiers(experiment)

    print("\nSTEP 1  Reading & appending GEOMETRY workbooks")
    geometry = load_and_append("Geom")
    print("\nSTEP 2  Renaming GEOMETRY columns to Python identifiers")
    geometry = rename_to_identifiers(geometry)

    print("\nSTEP 3  Building physical radius & chord distributions")
    geometry = add_physical_distributions(geometry)
    print(geometry[["blade_name", "adimensional_radius_r_r",
                    "adimensional_chord_c_r", "radius_distribution",
                    "chord_distribution"]].head(6).to_string(index=False))

    exp_out = os.path.join(PROCESSED, "experiment_all.csv")
    geo_out = os.path.join(PROCESSED, "geometry_all.csv")
    experiment.to_csv(exp_out, index=False)
    geometry.to_csv(geo_out, index=False)
    print(f"\nSaved {exp_out}  ({experiment.shape})")
    print(f"Saved {geo_out}  ({geometry.shape})")

    # Quick provenance summary
    print("\nEXPERIMENT unique propellers:", experiment["propeller_name"].nunique())
    print("GEOMETRY   unique blades    :", geometry["blade_name"].nunique())


if __name__ == "__main__":
    main()
```

02_solidity_analysis.py

```
 """
02_solidity_analysis.py  -  Week 2 (Data Science), Part 2
============================================================
Tasks addressed
---------------
1. For each propeller, compute the AREA OF EACH BLADE and the TOTAL blade area.
   Blade area is the definite integral of the chord along the radius; it is
   evaluated with the composite trapezoidal rule via numpy.trapz.
2. Compute the DISC AREA of every propeller:  A = pi * r^2  (r = D/2).
3. Compute propeller SOLIDITY = (total blade area) / (disc area).
4. COLLATE the solidity values with the experiment data.
5. Check whether solidity could be found for every propeller and describe the
   findings, with a supporting visualization.

Outputs:
   data/processed/solidity_table.csv     - one row per propeller
   data/processed/experiment_with_solidity.csv
   outputs/tables/solidity_table.csv
   outputs/figures/01_solidity_findings.png
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from config import PROCESSED, FIGURES, TABLES

plt.rcParams.update({"figure.dpi": 120, "font.size": 10})


def blade_area_trapz(sub):
    """Definite integral  A = integral of c(r) dr  over the blade span.

    Uses the composite trapezoidal rule (numpy.trapz) on the physical chord and
    radius distributions. Some blades in the raw data list each radial station
    more than once (repeated measurements); we average the chord per unique
    radius so the trapezoidal rule sees a clean, monotonically increasing span.
    Any NaN chords are linearly interpolated within the blade first, because the
    radial chord profile is smooth and integrating across a gap would bias the
    area low.
    """
    # collapse duplicate radial stations (mean chord per unique radius)
    sub = (sub.groupby("radius_distribution", as_index=False)["chord_distribution"]
              .mean()
              .sort_values("radius_distribution"))
    r = sub["radius_distribution"].to_numpy(dtype=float)
    c = sub["chord_distribution"].to_numpy(dtype=float)
    if len(r) < 2:
        return np.nan
    if np.isnan(c).any():
```

```
        good = ~np.isnan(c)
        if good.sum() < 2:
            return np.nan
        c = np.interp(r, r[good], c[good])   # fill interior gaps smoothly
    # numpy.trapz integrates y=c along x=r using the trapezoidal rule.
    return float(np.trapezoid(c, r)) if hasattr(np, "trapezoid") else float(np.trapz(c, r))


def main():
    geom = pd.read_csv(os.path.join(PROCESSED, "geometry_all.csv"))
    exp = pd.read_csv(os.path.join(PROCESSED, "experiment_all.csv"))

    # ---- 1. Area of each (single) blade, per blade name --------------------
    areas = (geom.groupby("blade_name")
                 .apply(blade_area_trapz, include_groups=False)
                 .rename("single_blade_area")
                 .reset_index())
    print("Single-blade areas computed for", len(areas), "blades")

    # ---- Propeller-level table: bring in blade count, diameter -------------
    prop_meta = (exp.groupby("propeller_name")
                    .agg(blade_name=("blade_name", "first"),
                         propeller_brand=("propeller_brand", "first"),
                         number_of_blades=("number_of_blades", "first"),
                         propeller_diameter=("propeller_diameter", "first"),
                         propeller_pitch=("propeller_pitch", "first"))
                    .reset_index())

    tab = prop_meta.merge(areas, on="blade_name", how="left")

    # ---- 1b. Total blade area = (#blades) x (single-blade area) ------------
    tab["total_blade_area"] = tab["number_of_blades"] * tab["single_blade_area"]

    # ---- 2. Disc area  A = pi * r^2 ----------------------------------------
    tab["disc_radius"] = tab["propeller_diameter"] / 2.0
    tab["disc_area"] = np.pi * tab["disc_radius"] ** 2

    # ---- 3. Solidity = total blade area / disc area ------------------------
    tab["solidity"] = tab["total_blade_area"] / tab["disc_area"]

    tab = tab.sort_values(["number_of_blades", "solidity"], na_position="last")
    tab.to_csv(os.path.join(PROCESSED, "solidity_table.csv"), index=False)
    tab.to_csv(os.path.join(TABLES, "solidity_table.csv"), index=False)

    # ---- 5. Findings: which propellers are missing solidity? --------------
    n_total = tab["propeller_name"].nunique()
    n_missing = int(tab["solidity"].isna().sum())
    missing_names = tab.loc[tab["solidity"].isna(), "propeller_name"].tolist()
    print("\n================ SOLIDITY FINDINGS ================")
    print(f"Propellers in experiment data : {n_total}")
    print(f"Solidity successfully computed : {n_total - n_missing}")
    print(f"Solidity MISSING (no geometry) : {n_missing}  -> {missing_names}")
    print(f"Solidity range  : {tab['solidity'].min():.4f} .. {tab['solidity'].max():.4f}")
    print(f"Solidity mean   : {tab['solidity'].mean():.4f}")
    print("By blade count (mean solidity):")
    print(tab.groupby('number_of_blades')['solidity'].mean().to_string())
```

```
    # ---- 4. Collate solidity onto the experiment data ---------------------
    exp_sol = exp.merge(tab[["propeller_name", "single_blade_area",
                             "total_blade_area", "disc_area", "solidity"]],
                        on="propeller_name", how="left")
    exp_sol.to_csv(os.path.join(PROCESSED, "experiment_with_solidity.csv"),
                   index=False)
    print(f"\nCollated experiment+solidity -> {exp_sol.shape}, "
          f"rows lacking solidity = {int(exp_sol['solidity'].isna().sum())}")

    # ---- Visualization: solidity findings ---------------------------------
    plotted = tab.dropna(subset=["solidity"])
    blade_levels = sorted(plotted["number_of_blades"].unique())

    fig, ax = plt.subplots(1, 3, figsize=(16, 5))

    # (a) coverage: how many propellers actually got a solidity value
    cov_labels = ["Solidity\ncomputed", "Solidity MISSING\n(no geometry)"]
    cov_vals = [n_total - n_missing, n_missing]
    ax[0].bar(cov_labels, cov_vals, color=["#4C72B0", "#C44E52"])
    for i, v in enumerate(cov_vals):
        ax[0].text(i, v + 2, str(v), ha="center", fontweight="bold")
    ax[0].set_ylabel("Number of propellers")
    ax[0].set_title(f"Solidity coverage\n{n_total-n_missing}/{n_total} propellers "
                    f"({100*(n_total-n_missing)/n_total:.0f}%)")

    # (b) distribution of computed solidity
    ax[1].hist(plotted["solidity"], bins=25, color="#4C72B0", edgecolor="white")
    ax[1].axvline(plotted["solidity"].mean(), color="red", ls="--",
                  label=f"mean={plotted['solidity'].mean():.3f}")
    ax[1].set_xlabel("Solidity  (total blade area / disc area)")
    ax[1].set_ylabel("Count")
    ax[1].set_title("Distribution of computed solidity")
    ax[1].legend()

    # (c) solidity vs blade count
    ax[2].boxplot([plotted.loc[plotted.number_of_blades == b, "solidity"]
                   for b in blade_levels],
                  tick_labels=[f"{b} blades" for b in blade_levels])
    ax[2].set_ylabel("Solidity")
    ax[2].set_title("Solidity by blade count")

    fig.suptitle(f"Solidity findings  ({n_missing} of {n_total} propellers had "
                 f"NO geometry → solidity N/A)", fontweight="bold")
    fig.tight_layout()
    fig.savefig(os.path.join(FIGURES, "01_solidity_findings.png"))
    print("\nSaved figure outputs/figures/01_solidity_findings.png")


if __name__ == "__main__":
    main()
```

03_eda_visualizations.py

```
 """
03_eda_visualizations.py  -  Week 2 (Data Science), Part 3
============================================================
Tasks addressed
---------------
1. Missing-value overview on the collated dataset.
2. Univariate analysis of the three performance outputs.
3. BIVARIATE analysis (the propeller performance curves and driver
   relationships) with written interpretation printed to the console.
4. Correlation HEATMAP between all numeric variables.

Outputs (outputs/figures/):
   02_target_distributions.png
   03_performance_curves.png
   04_bivariate_drivers.png
   05_correlation_heatmap.png
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

from config import PROCESSED, FIGURES

sns.set_theme(style="whitegrid", font_scale=0.95)
PAL = {2: "#4C72B0", 3: "#DD8452", 4: "#55A868"}


def main():
    df = pd.read_csv(os.path.join(PROCESSED, "experiment_with_solidity.csv"))

    # ------------------------------------------------------------------ #
    # 1. Univariate distributions of the three targets
    # ------------------------------------------------------------------ #
    targets = ["thrust_coefficient_output", "power_coefficient_output",
               "efficiency_output"]
    titles = ["Thrust coefficient  C_T", "Power coefficient  C_P",
              "Efficiency  eta"]
    fig, ax = plt.subplots(1, 3, figsize=(14, 4))
    for a, t, title in zip(ax, targets, titles):
        sns.histplot(df[t], kde=True, ax=a, color="#4C72B0")
        a.set_title(title)
        a.set_xlabel("")
    fig.suptitle("Univariate distributions of performance outputs",
                 fontweight="bold")
    fig.tight_layout()
    fig.savefig(os.path.join(FIGURES, "02_target_distributions.png"))
    plt.close(fig)
```

```
# ------------------------------------------------------------- #
# 2. Bivariate: the classic propeller performance curves vs J
# ------------------------------------------------------------- #
fig, ax = plt.subplots(1, 3, figsize=(15, 4.6))
for a, t, title in zip(ax, targets, titles):
    for name, sub in df.groupby("propeller_name"):
        sub = sub.sort_values("advanced_ratio_input")
        b = sub["number_of_blades"].iloc[0]
        a.plot(sub["advanced_ratio_input"], sub[t], lw=0.8, alpha=0.5,
               color=PAL.get(b, "gray"))
    a.set_xlabel("Advance ratio  J")
    a.set_title(title + "  vs  J")
# Efficiency explodes negative past zero-thrust (windmill state); clip the
# view to the physically meaningful working band so the classic
# rise-peak-fall efficiency curve is legible.
ax[2].set_ylim(-0.2, 0.95)
ax[0].set_ylabel("value")
handles = [plt.Line2D([0], [0], color=PAL[b], label=f"{b} blades")
           for b in (2, 3, 4)]
ax[2].legend(handles=handles, loc="upper right")
fig.suptitle("Bivariate analysis - performance curves vs advance ratio",
             fontweight="bold")
fig.tight_layout()
fig.savefig(os.path.join(FIGURES, "03_performance_curves.png"))
plt.close(fig)

# ------------------------------------------------------------- #
# 3. Bivariate drivers: solidity / RPM / diameter vs performance
# ------------------------------------------------------------- #
fig, ax = plt.subplots(1, 3, figsize=(15, 4.6))
sns.scatterplot(data=df, x="solidity", y="thrust_coefficient_output",
                hue="number_of_blades", palette=PAL, s=12, ax=ax[0],
                legend="full")
ax[0].set_title("C_T vs solidity")
sns.scatterplot(data=df, x="solidity", y="power_coefficient_output",
                hue="number_of_blades", palette=PAL, s=12, ax=ax[1],
                legend=False)
ax[1].set_title("C_P vs solidity")
sns.boxplot(data=df, x="number_of_blades", y="efficiency_output",
            hue="number_of_blades", palette=PAL, ax=ax[2], legend=False)
ax[2].set_title("Efficiency by blade count")
fig.suptitle("Bivariate analysis - geometric & operational drivers",
             fontweight="bold")
fig.tight_layout()
fig.savefig(os.path.join(FIGURES, "04_bivariate_drivers.png"))
plt.close(fig)

# ------------------------------------------------------------- #
# 4. Correlation heatmap
# ------------------------------------------------------------- #
num_cols = ["number_of_blades", "propeller_diameter", "propeller_pitch",
            "advanced_ratio_input", "rpm_rotation_input", "solidity",
            "total_blade_area", "disc_area",
            "thrust_coefficient_output", "power_coefficient_output",
            "efficiency_output"]
corr = df[num_cols].corr()
fig, a = plt.subplots(figsize=(10, 8))
```

```
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0,
                square=True, cbar_kws={"shrink": 0.8}, ax=a,
                annot_kws={"size": 7})
    a.set_title("Correlation heatmap of numeric variables", fontweight="bold")
    fig.tight_layout()
    fig.savefig(os.path.join(FIGURES, "05_correlation_heatmap.png"))
    plt.close(fig)

    # ------------------------------------------------------------------ #
    # Printed interpretation (goes into the write-up)
    # ------------------------------------------------------------------ #
    print("Missing values per column:")
    print(df.isna().sum()[df.isna().sum() > 0].to_string())
    print("\nCorrelation of drivers with efficiency:")
    print(corr["efficiency_output"].sort_values(ascending=False).round(2).to_string())
    print("\nCorrelation of drivers with thrust coefficient:")
    print(corr["thrust_coefficient_output"].sort_values(ascending=False).round(2).to_string())
    print("\nSaved 4 EDA figures to outputs/figures/")


if __name__ == "__main__":
    main()
```

04_ml_models.py

```
 """
04_ml_models.py  -  Week 2 (Machine Learning)
==============================================
Tasks addressed
---------------
1. Detect missing values and perform missing-value treatment.
2. Predict propeller performance (thrust coefficient C_T, power coefficient
   C_P, efficiency eta) from blade geometry + operational inputs, using the
   GRADIENT BOOSTING technique.
3. TRAIN on the 2-blade propellers, then EVALUATE on the "other" propellers
   (the held-out 3-blade and 4-blade propellers).
4. Build THREE model variants and compare them:
       (A) Without missing-value imputation  -> complete-case (drop rows whose
           solidity is missing).
       (B) With missing-value imputation      -> median-impute solidity.
       (C) Without solidity                    -> drop the solidity feature.

Where do the missing values come from?
   The experiment coefficients themselves are complete, but 106 of 240
   propellers have NO blade-geometry rows, so their SOLIDITY cannot be
   computed. After collation, ~48% of experiment rows have a missing solidity.
   Solidity is therefore the feature that drives the missing-value treatment.

Outputs:
   outputs/tables/model_comparison.csv
   outputs/figures/06_model_comparison_r2.png
   outputs/figures/07_pred_vs_actual.png
   outputs/figures/08_feature_importance.png
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

from config import PROCESSED, FIGURES, TABLES

TARGETS = {
    "thrust_coefficient_output": "C_T",
    "power_coefficient_output": "C_P",
    "efficiency_output": "eta",
}
# Base operational + geometric features. number_of_blades is intentionally
# EXCLUDED: the model trains only on 2-blade props (zero variance there) and
# must generalise to 3/4-blade props, so it cannot use blade count as an input.
BASE_FEATURES = ["propeller_diameter", "propeller_pitch",
                 "advanced_ratio_input", "rpm_rotation_input"]
```

```
SOLIDITY = "solidity"


def treat_efficiency_outliers(df):
    """Efficiency is only physically meaningful in the working state. Past the
    zero-thrust advance ratio the propeller windmills/brakes and eta = J*CT/CP
    explodes toward large negatives as CP -> 0. We clip the target to a
    physically sensible band [-1, 0.9]; CT and CP are left untouched (clean)."""
    out = df.copy()
    n_bad = int(((out["efficiency_output"] < -1) |
                 (out["efficiency_output"] > 0.9)).sum())
    out["efficiency_output"] = out["efficiency_output"].clip(-1.0, 0.9)
    print(f"  efficiency outlier treatment: clipped {n_bad} extreme rows to [-1, 0.9]")
    return out


def fit_eval(train, test, features, target, impute=False):
    """Fit a GradientBoostingRegressor and score it on the test frame.

    Returns (metrics dict, fitted model, imputer-or-None, y_true, y_pred)."""
    Xtr, ytr = train[features].copy(), train[target].values
    Xte, yte = test[features].copy(), test[target].values

    imputer = None
    if impute and SOLIDITY in features:
        imputer = SimpleImputer(strategy="median")
        Xtr[features] = imputer.fit_transform(Xtr[features])
        Xte[features] = imputer.transform(Xte[features])

    model = GradientBoostingRegressor(
        n_estimators=400, learning_rate=0.05, max_depth=3,
        subsample=0.9, random_state=42)
    model.fit(Xtr, ytr)
    pred = model.predict(Xte)
    metrics = {
        "R2": r2_score(yte, pred),
        "RMSE": np.sqrt(mean_squared_error(yte, pred)),
        "MAE": mean_absolute_error(yte, pred),
        "n_train": len(Xtr),
        "n_test": len(Xte),
    }
    return metrics, model, imputer, yte, pred


def main():
    df = pd.read_csv(os.path.join(PROCESSED, "experiment_with_solidity.csv"))

    # ---- 1. Missing-value audit ------------------------------------------
    print("STEP 1  Missing-value audit (collated experiment + solidity)")
    miss = df.isna().sum()
    print(miss[miss > 0].to_string())
    print(f"  rows with missing solidity: {int(df[SOLIDITY].isna().sum())} "
          f"/ {len(df)}  ({100*df[SOLIDITY].isna().mean():.1f}%)")

    df = treat_efficiency_outliers(df)

    # ---- Train / test split by blade count -------------------------------
```

```
train_all = df[df["number_of_blades"] == 2].copy()
test_all = df[df["number_of_blades"] != 2].copy()   # 3- and 4-blade props
print(f"\nSTEP 3  Split -> train (2 blades): {len(train_all)} rows / "
      f"{train_all['propeller_name'].nunique()} props | "
      f"test (3&4 blades): {len(test_all)} rows / "
      f"{test_all['propeller_name'].nunique()} props")

# Common test subset with KNOWN solidity => fair head-to-head for all 3
test_known = test_all.dropna(subset=[SOLIDITY]).copy()
print(f"  fair-comparison test subset (solidity known): {len(test_known)} rows")

# ---- 2 & 4. Three model variants ----------------------------------
variants = {
    "A. Without imputation (complete-case)":
        dict(features=BASE_FEATURES + [SOLIDITY], impute=False,
             drop_missing_train=True),
    "B. With imputation (median solidity)":
        dict(features=BASE_FEATURES + [SOLIDITY], impute=True,
             drop_missing_train=False),
    "C. Without solidity":
        dict(features=BASE_FEATURES, impute=False,
             drop_missing_train=False),
}

rows = []
stash = {}   # (variant, target) -> (y_true, y_pred, model, features)
for vname, cfg in variants.items():
    feats = cfg["features"]
    train = train_all.copy()
    if cfg["drop_missing_train"]:
        train = train.dropna(subset=[SOLIDITY])
    for target in TARGETS:
        m, model, _, yt, yp = fit_eval(
            train, test_known, feats, target, impute=cfg["impute"])
        m.update(variant=vname, target=TARGETS[target])
        rows.append(m)
        stash[(vname, target)] = (yt, yp, model, feats)

comp = pd.DataFrame(rows)[["variant", "target", "R2", "RMSE", "MAE",
                           "n_train", "n_test"]]
comp.to_csv(os.path.join(TABLES, "model_comparison.csv"), index=False)
print("\n================ MODEL COMPARISON (fair test subset) ============")
for tgt in TARGETS.values():
    print(f"\n  Target = {tgt}")
    sub = comp[comp.target == tgt]
    print(sub[["variant", "R2", "RMSE", "MAE", "n_train"]]
          .to_string(index=False))

# ---- Coverage note ------------------------------------------------
# The missing solidity sits in the TRAINING data (many 2-blade props have
# no geometry). Variant A discards those rows and trains on far fewer.
n_train_full = len(train_all)
n_train_A = len(train_all.dropna(subset=[SOLIDITY]))
print("\nTraining-data cost of each strategy:")
print(f"  A (complete-case) trains on {n_train_A} of {n_train_full} 2-blade rows "
      f"({100*n_train_A/n_train_full:.0f}%) - discards {n_train_full-n_train_A} rows "
      f"and, in deployment, cannot score any propeller lacking geometry.")
```

```
print(f"  B (imputation)    trains on {n_train_full} of {n_train_full} rows (100%).")
print(f"  C (no solidity)   trains on {n_train_full} of {n_train_full} rows (100%); "
      f"needs no geometry at all, so it can score every propeller.")

# ---- Figure 6: R2 comparison bar chart ------------------------------
fig, ax = plt.subplots(figsize=(11, 5.5))
pivot = comp.pivot(index="target", columns="variant", values="R2")
pivot = pivot.reindex(["C_T", "C_P", "eta"])
pivot.plot(kind="bar", ax=ax, width=0.75,
           color=["#4C72B0", "#55A868", "#DD8452"])
ax.set_ylabel("R^2 on held-out 3 & 4-blade propellers")
ax.set_xlabel("Target")
ax.set_title("Gradient Boosting - three missing-value strategies compared",
             fontweight="bold")
ax.axhline(0, color="k", lw=0.6)
ax.legend(fontsize=8, loc="lower left")
ax.tick_params(axis="x", rotation=0)
for c in ax.containers:
    ax.bar_label(c, fmt="%.2f", fontsize=7, padding=2)
fig.tight_layout()
fig.savefig(os.path.join(FIGURES, "06_model_comparison_r2.png"))
plt.close(fig)

# ---- Figure 7: predicted vs actual for variant B ---------------------
fig, ax = plt.subplots(1, 3, figsize=(15, 4.8))
vB = "B. With imputation (median solidity)"
for a, (target, short) in zip(ax, TARGETS.items()):
    yt, yp, _, _ = stash[(vB, target)]
    a.scatter(yt, yp, s=6, alpha=0.3, color="#4C72B0")
    lo, hi = min(yt.min(), yp.min()), max(yt.max(), yp.max())
    a.plot([lo, hi], [lo, hi], "r--", lw=1)
    r2 = r2_score(yt, yp)
    a.set_xlabel(f"actual {short}")
    a.set_ylabel(f"predicted {short}")
    a.set_title(f"{short}   (R^2 = {r2:.3f})")
fig.suptitle("Predicted vs actual - variant B (with imputation), "
             "held-out 3 & 4-blade props", fontweight="bold")
fig.tight_layout()
fig.savefig(os.path.join(FIGURES, "07_pred_vs_actual.png"))
plt.close(fig)

# ---- Figure 8: feature importance (variant B, all three targets) -----
fig, ax = plt.subplots(1, 3, figsize=(15, 4.5))
for a, (target, short) in zip(ax, TARGETS.items()):
    _, _, model, feats = stash[(vB, target)]
    imp = pd.Series(model.feature_importances_, index=feats).sort_values()
    a.barh(imp.index, imp.values, color="#4C72B0")
    a.set_title(f"Feature importance - {short}")
    a.tick_params(axis="y", labelsize=8)
fig.suptitle("Gradient Boosting feature importances (variant B)",
             fontweight="bold")
fig.tight_layout()
fig.savefig(os.path.join(FIGURES, "08_feature_importance.png"))
plt.close(fig)

print("\nSaved model_comparison.csv and figures 06-08.")
```

```
if __name__ == "__main__":
    main()
```

05_run_sql.py

```
 """
05_run_sql.py  -  Week 1 (SQL)
==============================
Loads the three raw experiment CSVs into an in-memory SQLite database
(experiment1 / experiment2 / experiment3) and answers the four Week-1 SQL
questions. The SQL itself lives in sql/week1_queries.sql; this runner executes
the key statements, prints the answers, and saves result tables/log.

Outputs:
   outputs/tables/sql_q2_efficiency_desc.csv
   outputs/tables/sql_q3_least100_power.csv
   outputs/tables/sql_answers.txt
   outputs/figures/09_sql_summary.png
"""
import os
import sqlite3
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from config import RAW, TABLES, FIGURES

COLMAP = {
    "Propeller's Name": "propeller_name",
    "Blade's Name": "blade_name",
    "Propeller's Brand": "propeller_brand",
    "Number of Blades": "number_of_blades",
    "Propeller's Diameter": "propeller_diameter",
    "Propeller's Pitch": "propeller_pitch",
    "Advanced Ratio Input": "advance_ratio",
    "RPM Rotation Input": "rpm",
    "Thrust Coefficient Output": "thrust_coeff",
    "Power Coefficient Output": "power_coeff",
    "Efficiency Output": "efficiency",
}


def build_db():
    con = sqlite3.connect(":memory:")
    for vol in (1, 2, 3):
        df = pd.read_csv(os.path.join(RAW, f"Experiment_vol{vol}.csv"))
        df = df.rename(columns=COLMAP)
        df.to_sql(f"experiment{vol}", con, index=False, if_exists="replace")
    return con


def main():
    con = build_db()
    log = []
```

```
def say(s):
    print(s)
    log.append(s)

say("=" * 68)
say("WEEK 1 - SQL ANSWERS")
say("=" * 68)

# Q1 ------------------------------------------------------------------
q1 = pd.read_sql("SELECT COUNT(DISTINCT propeller_name) AS n "
                 "FROM experiment1 WHERE thrust_coeff > 0.12", con)["n"][0]
q1_rows = pd.read_sql("SELECT COUNT(*) AS n FROM experiment1 "
                      "WHERE thrust_coeff > 0.12", con)["n"][0]
say("\nQ1. Experiment 1 - propellers with thrust coefficient > 12% (0.12):")
say(f"    distinct propellers : {q1}")
say(f"    operating-point rows: {q1_rows}")

# Q2 ------------------------------------------------------------------
q2 = pd.read_sql(
    "SELECT propeller_name, advance_ratio, rpm, thrust_coeff, power_coeff, "
    "efficiency FROM experiment1 ORDER BY efficiency DESC", con)
q2.to_csv(os.path.join(TABLES, "sql_q2_efficiency_desc.csv"), index=False)
say("\nQ2. Experiment 1 reordered by efficiency (descending). "
    f"Saved {len(q2)} rows. Top 5:")
say(q2.head(5).to_string(index=False))

# Q3 ------------------------------------------------------------------
q3 = pd.read_sql(
    "SELECT propeller_name, advance_ratio, rpm, power_coeff, efficiency "
    "FROM experiment1 ORDER BY power_coeff ASC LIMIT 100", con)
q3.to_csv(os.path.join(TABLES, "sql_q3_least100_power.csv"), index=False)
say("\nQ3. Experiment 1 - 100 least-performing rows by power coefficient. "
    "Saved 100 rows. Lowest 5:")
say(q3.head(5).to_string(index=False))

# Q4 ------------------------------------------------------------------
merged_sql = ("WITH merged AS (SELECT * FROM experiment1 UNION ALL "
              "SELECT * FROM experiment2 UNION ALL SELECT * FROM experiment3) ")
q4 = pd.read_sql(merged_sql + "SELECT COUNT(DISTINCT propeller_name) AS n "
                 "FROM merged WHERE efficiency <= 0", con)["n"][0]
q4_rows = pd.read_sql(merged_sql + "SELECT COUNT(*) AS n FROM merged "
                      "WHERE efficiency <= 0", con)["n"][0]
total_props = pd.read_sql(merged_sql + "SELECT COUNT(DISTINCT propeller_name)"
                          " AS n FROM merged", con)["n"][0]
say("\nQ4. Merged experiment 1+2+3 - propellers with efficiency <= 0:")
say(f"    distinct propellers : {q4}  (of {total_props} total)")
say(f"    operating-point rows: {q4_rows}")

with open(os.path.join(TABLES, "sql_answers.txt"), "w") as f:
    f.write("\n".join(log))

# ---- Figure 9: visual summary of the SQL answers ---------------------
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ax[0].axis("off")
text = (
    "WEEK 1  -  SQL ANSWERS\n"
    "-----------------------------------------\n\n"
```

```
        f"Q1  Experiment 1, thrust coeff > 12%\n"
        f"      distinct propellers : {q1}\n"
        f"      rows                : {q1_rows}\n\n"
        f"Q2  Experiment 1 sorted by efficiency DESC\n"
        f"      {len(q2)} rows  (see CSV)\n\n"
        f"Q3  100 least by power coefficient\n"
        f"      lowest CP = {q3['power_coeff'].min():.4f}\n\n"
        f"Q4  Merged 1+2+3, efficiency <= 0\n"
        f"      distinct propellers : {q4} of {total_props}\n"
        f"      rows                : {q4_rows}"
    )
    ax[0].text(0.02, 0.98, text, va="top", ha="left", family="monospace",
               fontsize=11)

    top10 = q2.head(10)
    ax[1].barh(range(len(top10))[::-1], top10["efficiency"], color="#4C72B0")
    ax[1].set_yticks(range(len(top10))[::-1])
    ax[1].set_yticklabels(top10["propeller_name"], fontsize=7)
    ax[1].set_xlabel("Efficiency")
    ax[1].set_title("Q2 - top 10 rows by efficiency (Experiment 1)")
    fig.suptitle("SQL task summary", fontweight="bold")
    fig.tight_layout()
    fig.savefig(os.path.join(FIGURES, "09_sql_summary.png"))
    print("\nSaved SQL result tables and outputs/figures/09_sql_summary.png")


if __name__ == "__main__":
    main()
```

06_tableau_dashboard.py

```
 """
06_tableau_dashboard.py  -  Week 2 (Tableau / data storytelling)
================================================================
Two deliverables for the Tableau task:

1. A clean Tableau EXTRACT  (data/processed/tableau_extract.csv) enriched with
   the fields a business dashboard needs: pitch-to-diameter ratio, working-state
   flag, clipped efficiency, per-propeller peak efficiency and the advance ratio
   at which it occurs, blade class, and solidity.

2. A rendered, storytelling DASHBOARD image (outputs/figures/10_dashboard.png)
   that mirrors the Tableau dashboard described in report/tableau_dashboard_spec.md
   - a KPI banner plus five coordinated views that walk a reader from "what do we
   have" to "what drives efficiency" to "the design trade-off".
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

from config import PROCESSED, FIGURES

BLUE, ORANGE, GREEN, RED = "#4C72B0", "#DD8452", "#55A868", "#C44E52"


def build_extract():
    df = pd.read_csv(os.path.join(PROCESSED, "experiment_with_solidity.csv"))
    df["pitch_to_diameter"] = df["propeller_pitch"] / df["propeller_diameter"]
    df["working_state"] = np.where(df["thrust_coefficient_output"] > 0,
                                   "Working (thrust>0)", "Windmill/brake")
    df["efficiency_clipped"] = df["efficiency_output"].clip(-1.0, 0.9)
    df["blade_class"] = df["number_of_blades"].astype(int).astype(str) + "-blade"

    # per-propeller peak efficiency in the working state + where it occurs
    work = df[df["thrust_coefficient_output"] > 0]
    peak = (work.sort_values("efficiency_output")
                .groupby("propeller_name")
                .tail(1)[["propeller_name", "efficiency_output",
                          "advanced_ratio_input"]]
                .rename(columns={"efficiency_output": "peak_efficiency",
                                 "advanced_ratio_input": "j_at_peak_eff"}))
    df = df.merge(peak, on="propeller_name", how="left")

    out = os.path.join(PROCESSED, "tableau_extract.csv")
    df.to_csv(out, index=False)
    print(f"Saved Tableau extract -> {out}  {df.shape}")
    return df
```

```
def render_dashboard(df):
    prop = (df.groupby("propeller_name")
              .agg(brand=("propeller_brand", "first"),
                   blades=("number_of_blades", "first"),
                   diameter=("propeller_diameter", "first"),
                   pitch=("propeller_pitch", "first"),
                   pd_ratio=("pitch_to_diameter", "first"),
                   solidity=("solidity", "first"),
                   peak_eff=("peak_efficiency", "first"))
              .reset_index())

    fig = plt.figure(figsize=(16, 11))
    fig.patch.set_facecolor("white")
    gs = gridspec.GridSpec(3, 3, height_ratios=[0.55, 1, 1],
                           hspace=0.42, wspace=0.28,
                           left=0.06, right=0.97, top=0.9, bottom=0.06)

    fig.suptitle("UAV Propeller Performance - Executive Dashboard",
                 fontsize=19, fontweight="bold", y=0.965)
    fig.text(0.5, 0.925,
             "Which propellers deliver the most efficient thrust, and what "
             "geometry drives it?  |  240 propellers, 27,495 operating points",
             ha="center", fontsize=11, color="#555")

    # ---- KPI banner ----------------------------------------------------
    kpis = [
        ("Propellers tested", f"{prop['propeller_name'].nunique()}", BLUE),
        ("Operating points", f"{len(df):,}", BLUE),
        ("Best peak efficiency", f"{prop['peak_eff'].max():.2f}", GREEN),
        ("Median peak efficiency", f"{prop['peak_eff'].median():.2f}", ORANGE),
        ("Median solidity", f"{prop['solidity'].median():.3f}", BLUE),
    ]
    kpi_gs = gridspec.GridSpecFromSubplotSpec(1, 5, subplot_spec=gs[0, :],
                                              wspace=0.15)
    for i, (label, value, color) in enumerate(kpis):
        a = fig.add_subplot(kpi_gs[0, i])
        a.axis("off")
        a.add_patch(plt.Rectangle((0, 0), 1, 1, transform=a.transAxes,
                                  facecolor=color, alpha=0.12))
        a.text(0.5, 0.62, value, ha="center", va="center", fontsize=22,
               fontweight="bold", color=color)
        a.text(0.5, 0.22, label, ha="center", va="center", fontsize=10,
               color="#444")

    # ---- View 1: efficiency envelope vs advance ratio --------------------
    a1 = fig.add_subplot(gs[1, 0])
    for _, sub in df.groupby("propeller_name"):
        sub = sub.sort_values("advanced_ratio_input")
        a1.plot(sub["advanced_ratio_input"], sub["efficiency_clipped"],
                color=BLUE, lw=0.4, alpha=0.15)
    a1.set_ylim(0, 0.9)
    a1.set_xlim(0, 1.6)
    a1.set_xlabel("Advance ratio  J")
    a1.set_ylabel("Efficiency")
    a1.set_title("1) Efficiency rises, peaks, then falls with advance ratio",
                 fontsize=11, fontweight="bold")
```

```
    # ---- View 2: peak efficiency by brand (top 12) ----------------------
    a2 = fig.add_subplot(gs[1, 1])
    brand_eff = (prop.groupby("brand")["peak_eff"].mean()
                     .sort_values(ascending=False).head(12))
    a2.barh(brand_eff.index[::-1], brand_eff.values[::-1], color=BLUE)
    a2.set_xlabel("Mean peak efficiency")
    a2.set_title("2) Peak efficiency by brand (top 12)",
                 fontsize=11, fontweight="bold")
    a2.tick_params(axis="y", labelsize=8)

    # ---- View 3: pitch/diameter vs peak efficiency ----------------------
    a3 = fig.add_subplot(gs[1, 2])
    cmap = {2: BLUE, 3: ORANGE, 4: GREEN}
    for b in sorted(prop["blades"].unique()):
        s = prop[prop["blades"] == b]
        a3.scatter(s["pd_ratio"], s["peak_eff"], s=22, alpha=0.7,
                   color=cmap[b], label=f"{b}-blade")
    a3.set_xlabel("Pitch / Diameter")
    a3.set_ylabel("Peak efficiency")
    a3.set_title("3) Higher pitch/diameter -> higher peak efficiency",
                 fontsize=11, fontweight="bold")
    a3.legend(fontsize=8)

    # ---- View 4: solidity vs peak efficiency ----------------------------
    a4 = fig.add_subplot(gs[2, 0])
    sol = prop.dropna(subset=["solidity"])
    for b in sorted(sol["blades"].unique()):
        s = sol[sol["blades"] == b]
        a4.scatter(s["solidity"], s["peak_eff"], s=22, alpha=0.7,
                   color=cmap[b], label=f"{b}-blade")
    a4.set_xlabel("Solidity")
    a4.set_ylabel("Peak efficiency")
    a4.set_title("4) Solidity vs efficiency (134 props with geometry)",
                 fontsize=11, fontweight="bold")
    a4.legend(fontsize=8)

    # ---- View 5: thrust vs power trade-off (density) --------------------
    a5 = fig.add_subplot(gs[2, 1])
    work = df[df["thrust_coefficient_output"] > 0]
    hb = a5.hexbin(work["power_coefficient_output"],
                   work["thrust_coefficient_output"], gridsize=35,
                   cmap="Blues", mincnt=1)
    a5.set_xlabel("Power coefficient  C_P")
    a5.set_ylabel("Thrust coefficient  C_T")
    a5.set_title("5) Thrust-power trade-off (working state)",
                 fontsize=11, fontweight="bold")
    fig.colorbar(hb, ax=a5, shrink=0.8, label="rows")

    # ---- View 6: efficiency by blade class ------------------------------
    a6 = fig.add_subplot(gs[2, 2])
    order = sorted(prop["blades"].unique())
    data = [prop.loc[prop["blades"] == b, "peak_eff"].dropna() for b in order]
    bp = a6.boxplot(data, tick_labels=[f"{b}-blade" for b in order],
                    patch_artist=True)
    for patch, b in zip(bp["boxes"], order):
        patch.set_facecolor(cmap[b])
        patch.set_alpha(0.7)
```

```
    a6.set_ylabel("Peak efficiency")
    a6.set_title("6) Peak efficiency by blade count",
                 fontsize=11, fontweight="bold")

    fig.savefig(os.path.join(FIGURES, "10_dashboard.png"), dpi=110,
                facecolor="white")
    plt.close(fig)
    print("Saved dashboard -> outputs/figures/10_dashboard.png")


def main():
    df = build_extract()
    render_dashboard(df)


if __name__ == "__main__":
    main()
```

07_build_submission.py

```
 """
07_build_submission.py
======================
Package the three deliverables into submission-ready formats
(pdf / png / zip):

  submission/1_Writeup_Propeller_Performance.pdf   <- report/REPORT.md + figures
  submission/2_Screenshots.zip                     <- the 10 PNG figures
  submission/3_SourceCode.pdf                       <- all .py + .sql, readable
  submission/Propeller_Capstone_Full_Project.zip    <- whole runnable project

Rendering uses the pre-installed Chromium via Playwright (HTML -> PDF), so the
PDFs are self-contained (figures embedded as base64).
"""
import base64
import glob
import html
import os
import re
import zipfile

import markdown as md_lib
from playwright.sync_api import sync_playwright

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, ".."))
REPORT_MD = os.path.join(ROOT, "report", "REPORT.md")
FIG_DIR = os.path.join(ROOT, "outputs", "figures")
SUB = os.path.join(ROOT, "submission")
os.makedirs(SUB, exist_ok=True)

CHROMIUM = glob.glob("/opt/pw-browsers/chromium-*/chrome-linux/chrome")
CHROMIUM = CHROMIUM[0] if CHROMIUM else None

CSS = """
<style>
  @page { size: A4; margin: 18mm 16mm; }
  body { font-family: 'Helvetica Neue', Arial, sans-serif; color:#1a1a1a;
         font-size: 11.5px; line-height: 1.5; }
  h1 { font-size: 22px; color:#1F3A5F; border-bottom:3px solid #4C72B0;
       padding-bottom:6px; }
  h2 { font-size: 16px; color:#1F3A5F; margin-top:22px;
       border-bottom:1px solid #ccd; padding-bottom:3px; }
  h3 { font-size: 13px; color:#345; }
  table { border-collapse: collapse; width: 100%; margin: 10px 0; font-size:10.5px; }
  th,td { border:1px solid #bbc; padding:5px 8px; text-align:left; }
  th { background:#eef2f8; }
  code { background:#f2f4f7; padding:1px 4px; border-radius:3px;
         font-family: 'SFMono-Regular', Consolas, monospace; font-size:10.5px; }
  pre { background:#f6f8fa; padding:10px; border-radius:6px; overflow-x:auto;
        border:1px solid #e1e4e8; }
  pre code { background:none; }
```

```
  blockquote { border-left:4px solid #DD8452; margin:10px 0; padding:4px 12px;
               background:#fff8f3; color:#443; }
  img.figure { width:100%; max-width:720px; display:block; margin:10px auto 4px;
               border:1px solid #ddd; border-radius:4px; }
  .cap { text-align:center; font-size:10px; color:#666; margin-bottom:16px; }
</style>
"""


def data_uri(path):
    with open(path, "rb") as f:
        b64 = base64.b64encode(f.read()).decode()
    return f"{b64}"


def build_report_html():
    with open(REPORT_MD) as f:
        text = f.read()
    body = md_lib.markdown(text, extensions=["tables", "fenced_code"])

    # Inline each figure right after its first mention, whether that mention is
    # bare (`NN_name.png`) or path-prefixed (`outputs/figures/NN_name.png`).
    figs = sorted(f for f in os.listdir(FIG_DIR) if f.endswith(".png"))
    embedded = set()
    for fig in figs:
        uri = data_uri(os.path.join(FIG_DIR, fig))
        img = (f'<img class="figure" src="{uri}"/>'
               f'<div class="cap">Figure &mdash; {fig}</div>')
        pattern = re.compile(r"(<code>(?:[^<]*/)?" + re.escape(fig) + r"</code>)")
        new_body, n = pattern.subn(r"\1" + img, body, count=1)
        if n:
            body = new_body
            embedded.add(fig)

    # Safety net: any figure not referenced inline is appended as a gallery so
    # every simulation screenshot appears in the PDF.
    missing = [f for f in figs if f not in embedded]
    if missing:
        gallery = ['<h2 style="page-break-before:always">Appendix &mdash; '
                   'additional figures</h2>']
        for fig in missing:
            uri = data_uri(os.path.join(FIG_DIR, fig))
            gallery.append(f'<img class="figure" src="{uri}"/>'
                           f'<div class="cap">Figure &mdash; {fig}</div>')
        body += "".join(gallery)

    return f"<!doctype html><html><head><meta charset='utf-8'>{CSS}</head>" \
           f"<body>{body}</body></html>"


def build_sourcecode_html():
    files = sorted(glob.glob(os.path.join(HERE, "*.py"))) + \
            [os.path.join(HERE, "sql", "week1_queries.sql")]
    order = ["config.py", "01_data_preparation.py", "02_solidity_analysis.py",
             "03_eda_visualizations.py", "04_ml_models.py", "05_run_sql.py",
             "06_tableau_dashboard.py", "07_build_submission.py", "run_all.py",
             "week1_queries.sql"]
```

```
    files = sorted(files, key=lambda p: order.index(os.path.basename(p))
                   if os.path.basename(p) in order else 99)

    parts = ["<h1>UAV Propeller Performance &mdash; Source Code</h1>",
             "<p>All analysis, machine-learning and SQL source for the capstone. "
             "Files are listed in execution order.</p>",
             "<h2>Contents</h2><ol>"]
    for p in files:
        parts.append(f"<li>{html.escape(os.path.basename(p))}</li>")
    parts.append("</ol>")
    for p in files:
        name = os.path.basename(p)
        with open(p) as f:
            code = f.read()
        parts.append(f'<h2 style="page-break-before:always">{html.escape(name)}</h2>')
        parts.append(f"<pre><code>{html.escape(code)}</code></pre>")
    return f"<!doctype html><html><head><meta charset='utf-8'>{CSS}</head>" \
           f"<body>{''.join(parts)}</body></html>"


def html_to_pdf(html_str, out_pdf):
    with sync_playwright() as p:
        browser = p.chromium.launch(executable_path=CHROMIUM)
        page = browser.new_page()
        page.set_content(html_str, wait_until="networkidle")
        page.pdf(path=out_pdf, format="A4", print_background=True,
                 margin={"top": "16mm", "bottom": "16mm",
                         "left": "14mm", "right": "14mm"})
        browser.close()


def zip_screenshots():
    out = os.path.join(SUB, "2_Screenshots.zip")
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for fig in sorted(os.listdir(FIG_DIR)):
            if fig.endswith(".png"):
                z.write(os.path.join(FIG_DIR, fig), fig)
        # include the propeller reference diagram too
        diag = os.path.join(ROOT, "report", "propeller_diagram.png")
        if os.path.exists(diag):
            z.write(diag, "00_propeller_reference_diagram.png")
    print("Wrote", out)


def zip_full_project():
    out = os.path.join(SUB, "Propeller_Capstone_Full_Project.zip")
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for base, _, names in os.walk(ROOT):
            if "/submission" in base or "/.git" in base:
                continue
            for n in names:
                fp = os.path.join(base, n)
                z.write(fp, os.path.relpath(fp, ROOT))
    print("Wrote", out)


def main():
```

```
    print("Rendering write-up PDF ...")
    html_to_pdf(build_report_html(),
                os.path.join(SUB, "1_Writeup_Propeller_Performance.pdf"))
    print("Rendering source-code PDF ...")
    html_to_pdf(build_sourcecode_html(),
                os.path.join(SUB, "3_SourceCode.pdf"))
    print("Zipping screenshots ...")
    zip_screenshots()
    print("Zipping full project ...")
    zip_full_project()
    print("\nSubmission files:")
    for f in sorted(os.listdir(SUB)):
        size = os.path.getsize(os.path.join(SUB, f)) / 1024
        print(f"  {f:45s} {size:8.1f} KB")


if __name__ == "__main__":
    main()
```

run_all.py

```
 """Run the full propeller-performance pipeline end to end."""
import runpy
import os

HERE = os.path.dirname(os.path.abspath(__file__))
STEPS = [
    "01_data_preparation.py",
    "02_solidity_analysis.py",
    "03_eda_visualizations.py",
    "04_ml_models.py",
    "05_run_sql.py",
    "06_tableau_dashboard.py",
]

for step in STEPS:
    print("\n" + "#" * 72 + f"\n# RUN {step}\n" + "#" * 72)
    runpy.run_path(os.path.join(HERE, step), run_name="__main__")

print("\nPipeline complete. See ../outputs/figures and ../outputs/tables.")
```

week1_queries.sql

```
-- ============================================================================
-- Week 1 - SQL tasks (UAV Propeller Performance)
-- ----------------------------------------------------------------------------
-- Tables (loaded by run_sql.py from the raw experiment CSVs):
--   experiment1  <- Experiment_vol1.csv
--   experiment2  <- Experiment_vol2.csv
--   experiment3  <- Experiment_vol3.csv
-- Column names are the original labels, exposed to SQL as safe aliases:
--   propeller_name, blade_name, propeller_brand, number_of_blades,
--   propeller_diameter, propeller_pitch, advance_ratio, rpm,
--   thrust_coeff, power_coeff, efficiency
-- Thrust coefficient is a fraction, so "more than 12%" means > 0.12.
-- ============================================================================


-- Q1. In experiment 1, total number of propellers whose thrust coefficient
--     output is more than 12% (i.e. > 0.12). A "propeller" is one distinct
--     Propeller's Name; we count the distinct propellers that reach CT > 0.12.
SELECT COUNT(DISTINCT propeller_name) AS propellers_ct_over_12pct
FROM   experiment1
WHERE  thrust_coeff > 0.12;

-- (companion) how many individual operating-point ROWS exceed 12% thrust:
SELECT COUNT(*) AS rows_ct_over_12pct
FROM   experiment1
WHERE  thrust_coeff > 0.12;


-- Q2. Reorder experiment 1 in descending order of efficiency output.
--     (Preview of the top of the reordered table.)
SELECT propeller_name, advance_ratio, rpm,
       thrust_coeff, power_coeff, efficiency
FROM   experiment1
ORDER BY efficiency DESC;


-- Q3. The 100 least-performing propellers based on power coefficient output
--     in experiment 1 (the 100 rows with the smallest power coefficient).
SELECT propeller_name, advance_ratio, rpm, power_coeff, efficiency
FROM   experiment1
ORDER BY power_coeff ASC
LIMIT 100;


-- Q4. Merge experiment 1, 2 and 3, then count the propellers whose efficiency
--     output is negative OR zero. UNION ALL stacks the three databases; we
--     count the distinct propellers that ever record efficiency <= 0.
WITH merged AS (
    SELECT * FROM experiment1
    UNION ALL
    SELECT * FROM experiment2
    UNION ALL
```

```
    SELECT * FROM experiment3
)
SELECT COUNT(DISTINCT propeller_name) AS propellers_eff_le_0
FROM   merged
WHERE  efficiency <= 0;

-- (companion) total ROWS across the merged data with efficiency <= 0:
WITH merged AS (
    SELECT * FROM experiment1
    UNION ALL
    SELECT * FROM experiment2
    UNION ALL
    SELECT * FROM experiment3
)
SELECT COUNT(*) AS rows_eff_le_0
FROM   merged
WHERE  efficiency <= 0;
```